
qlib Integration

Return & risk prediction, portfolio optimization, and score enhancement

AI Analyst embeds Microsoft `qlib` as a lean, in-process library to power its quantitative desk: a machine-learned return forecast, a factor-structured forward covariance, portfolio optimization, and a walk-forward backtest, with the learned signal folded into the idea scanner and the investment committee. This volume documents the *engineering* of that integration — the lean-embed principle, the module map, the Postgres-to-qlib panel adapter, the dual optimizer dispatcher, the committee and scanner wiring, the REST surface, and operations. The *mathematics* of each model is specified in the *Quantitative & ML Models* volume.

Contents

1	What the integration delivers	2
2	How qlib maps onto the desk's needs	2
3	Module map	3
4	The panel adapter	3
5	Dual optimizer dispatch	3
6	Score enhancement: alpha + regime-IC weights	4
7	Where the macro-regime IC feeds forecasting	4
8	Committee integration	4
9	Agentic alpha research (LangGraph)	5
9.1	Module map	5
9.2	Graph topology	5
9.3	State and reducers	6
9.4	Routing and termination	7
9.5	The agents	7
9.6	Two hard boundaries	7
9.7	Background job and ledger	8
9.8	Promotion	8
10	REST API and frontend	8
11	Operations	9

1 What the integration delivers

The quantitative desk adds a machine-learned layer on top of the deterministic valuation engine and the fundamentals warehouse:

- **Return prediction** — a cross-sectional LightGBM alpha model over the monthly fundamentals panel, producing a per-name expected forward return (the `mu` consumed by the optimizer and the scanner).
- **Risk prediction** — a factor-structured forward covariance (`StructuredCovEstimator`), $\Sigma = F \Sigma_B F^T + \text{diag}(\mathbf{d})$, with its decomposition exposed to the enhanced-indexing optimizer.
- **Portfolio optimization** — both a native optimizer *and* qlib's optimizers, selectable at runtime.
- **Score enhancement** — the learned alpha plus regime-conditioned factor-IC weights blended into the value + sentiment interest score.
- **Committee integration** — a `quant_signals` evidence node and a Quantitative Factor Analyst specialist.
- **Agentic model research** — a LangGraph loop in which a quantitative researcher, a model validation unit, a portfolio manager and an external advisor iterate over the training specification, producing a validation report per round and gating promotion on out-of-sample improvement (§9).
- **REST + UI** — a `/api/quant/*` router and a *Quant* tab.

Everything degrades gracefully: with no trained model, or with qlib or an optional dependency absent, each surface returns a defined fallback and the application runs unchanged.

The lean-embed principle

qlib's model, risk estimator, optimizers, and evaluation run as *plain libraries* on a MultiIndex DataFrame the app builds from Postgres. The integration never uses qlib's expression engine, its `.bin` data provider, the Alpha158/360 handlers, or the recorder/`qrun` workflow — so there is no `qlib.init`, no data-provider bootstrap, and no mlflow tracking backend. Only `lightgbm`, `scikit-learn`, and `cvxpy` become hard new runtime dependencies.

2 How qlib maps onto the desk's needs

Need	qlib capability used	Module
Return prediction	LGBModel + DatasetH + DataHandlerLP.from_df	api/quant/qlib_alpha.py
Risk prediction	StructuredCovEstimator	api/quant/qlib_risk.py
Portfolio optimization	PortfolioOptimizer, EnhancedIndexingOptimizer	api/quant/qlib_optimize.py
Evaluation / backtest	calc_ic, risk_analysis	api/quant/qlib_backtest.py

3 Module map

File	Responsibility
<code>qlib_data.py</code>	Build the monthly cross-sectional feature/label panel from Postgres (reuses the regime-IC loaders and point-in-time alignment).
<code>qlib_alpha.py</code>	AlphaLGB model (recorder-free), <code>train/predict/evaluate</code> , persistence, lazy cache, CLI.
<code>qlib_risk.py</code>	<code>structured_cov()</code> → annualized Σ + the $(F, \Sigma_B, \mathbf{d})$ decomposition; sync price/returns loader.
<code>qlib_optimize.py</code>	<code>solve(optimizer, ...)</code> dispatcher over the native + five qlib backends → a uniform <code>PortfolioSolution</code> .
<code>qlib_backtest.py</code>	Walk-forward out-of-sample signal backtest scored with <code>qlib_risk_analysis</code> .
<code>alpha_signal.py</code>	Central bridge: cached latest cross-section + per-ticker expected returns (used by scanner, committee, router).
<code>ic_weights.py</code>	Regime-conditioned factor-IC → per-family scoring weights.
<code>optimize.py</code> / <code>risk.py</code>	Native SLSQP/MIP optimizer; sample / Ledoit–Wolf / FF structured covariance.
<code>research/</code>	Agentic research package: the LangGraph loop, the quality-attribute battery, the robustness rating, the ledger, and the dossier (§9).
<code>routers/quant.py</code>	<code>/api/quant/{backends,alpha,risk,optimize,backtest,retrain,research/*}</code> .

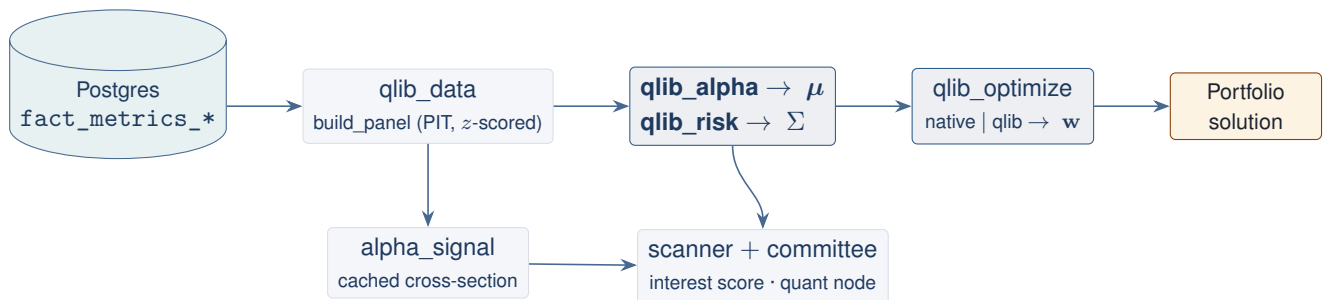


Figure 1. The lean qlib embed. One point-in-time panel feeds the alpha and risk models; the optimizer dispatcher consumes their μ and Σ ; a cached bridge exposes the alpha to the scanner and committee.

4 The panel adapter

`qlib_data.build_panel` produces a qlib-style `MultiIndex` frame — rows (`datetime, instrument`), columns (`feature|label`) — consumed directly by `DataHandlerLP.from_df`. Features are importance-ranked fundamentals from four families, cross-sectionally z -scored per month; the label is the forward 1m/3m return; fundamentals are aligned point-in-time (`period_end + 90d` → month-end). This is the *same* loader and alignment the regime-IC research uses, so the production forecast and the research signal share one lookahead-safe data path. The alpha model trains through the real qlib `DatasetH` pipeline; the `AlphaLGB` subclass attaches `LightGBM`’s callbacks directly in `fit`, so training runs with plain `LightGBM` and needs no `qlib.init` or tracking backend. Prediction runs the stored booster on any reindexed feature matrix.

5 Dual optimizer dispatch

`qlib_optimize.solve(optimizer, tickers, mu, sigma, ...)` dispatches to either backend family and returns a uniform `PortfolioSolution` (weights, annualized expected return / vol / Sharpe, factor exposures, diagnostics):

optimizer	Engine	Notes
native	optimize.py (SLSQP / cvxpy-MIP)	Full constraints: sector caps, vol cap, cardinality, turnover.
qlib_mvo / _gmw / _rp / _inv	qlib PortfolioOptimizer	Mean-variance / min-variance / risk-parity / inverse-vol; long-only, fully invested.
qlib_enhanced_indexing	qlib EnhancedIndexingOptimizer	Benchmark-relative tracking-error; consumes the StructuredCovEstimator decomposition.

Both families are fed the same annualized μ (alpha or historical mean) and Σ (qlib-structured, Ledoit–Wolf, or sample). Neither is a default-only path — both ship enabled and are chosen per request or from the UI selector.

6 Score enhancement: alpha + regime-IC weights

The value + sentiment interest score (`screener_agent`) blends two learned inputs on top of the fundamental composite:

1. **Normalized alpha** v_α — the qlib model’s expected return, cross-sectionally normalized, as the dominant term (default weight 0.40) when the model covers the shortlist.
2. **Regime-IC family weights** — the fundamental score budget (value / growth) is re-split by how predictive each family has been *in the current macro regime* (§7), rather than by fixed constants.

When no trained model or IC data is available the score reduces exactly to the fixed-weight composite (guaranteed by test). Each row reports `alpha`, `alpha_percentile`, and a `score_components` breakdown; the response carries a `scoring` block naming the weights and model actually used.

7 Where the macro-regime IC feeds forecasting

The macro-cycle model (HMM / VAE) labels each month with a regime; the regime-IC engine then computes, *conditioned on that regime*, the Spearman rank IC of every fundamental metric against forward returns, into `fact_equity_factor_ic_regime`. The one live consumption path in the desk is scoring: `ic_weights.family_weights` reads the latest regime’s rows, aggregates per-metric |IC| to family-level emphasis, and the scanner re-splits its fundamental budget accordingly — so the interest score leans harder on whichever factor family the current regime says is most predictive.

Scope

The LightGBM alpha model itself is regime-*agnostic*: it learns from factor *values* (including momentum/volatility from the market-factor family), and the regime enters forecasting through the score weights, not as a model feature. The committee receives macro regime context from the macro node and the `quant_signals` block, but not the factor-IC table directly.

8 Committee integration

- **State** — `InvestmentCommitteeState.quant_signals` plus a config toggle `enable_quant_signals`.
- **Evidence node** — `qlib_signals_node` runs in the *prepare* phase (once, shared across providers): the ticker’s alpha expected return, universe percentile, and model rank-IC, plus a peer-group factor-risk read (forward vol, factor exposures, minimum-variance weight). It auto-fans-out from the engine and auto-fans-in to every analyst, and degrades to `available: false` on any error, so it never fails a run.

- **Specialist** — the Quantitative Factor Analyst reconciles the statistical alpha/risk signals with the fundamental/DCF thesis and flags alpha-vs-valuation disagreements.

9 Agentic alpha research (LangGraph)

Code: `api/quant/research/`. The one-shot trainer fits one model on one fixed parametrization and promotes whatever comes out. This package replaces that with a critiqued search: four agents iterate over the training specification, each round emits a full model validation report, and promotion is gated on beating the incumbent out-of-sample.

Why a loop, and why critics

A single fit has no mechanism for noticing that its own number is untrustworthy. On the shipped US `forward_12m` model the purged walk-forward rank-IC of the baseline specification is *negative* (-0.013); two preprocessing changes — median imputation and label winsorization — move it to $+0.017$. Neither change is exotic, and neither would be tried by a pipeline with no critique step. The critics exist because a search will otherwise optimize the headline: a model can improve its rank-IC purely by discarding a third of the universe.

9.1 Module map

File	Responsibility
<code>spec.py</code>	<code>TrainingSpec</code> — the searched parametrization — plus <code>apply_patch</code> , the whitelisting/clamping validator, and <code>describe_search_space</code> (the lecture the researcher is taught from).
<code>preprocess.py</code>	Sample selection, winsorization, robust/rank normalization, imputation, sector / size neutralization, purged splitting.
<code>models.py</code>	Six model families behind one <code>.model.predict(ndarray)</code> contract; the ensemble wrapper.
<code>refdata.py</code>	Point-in-time GICS, market cap, and Fama–French betas — the maps the breakdowns and the rating group by.
<code>evaluate.py</code>	Purged walk-forward scoring and the quality-attribute battery; the shared <code>breakdown()</code> .
<code>perturb.py</code>	The perturbation battery, industry deconfounding, and the ordinal robustness rating.
<code>report.py</code>	The iteration report, and the allowlisted, leakage-swept packet the agents are shown.
<code>schemas.py</code> / <code>prompts.py</code>	Pydantic structured outputs; the four personas written as lectures.
<code>nodes.py</code> / <code>graph.py</code>	Node functions; graph assembly, <code>run_research</code> , and the CLI.
<code>runner.py</code>	Background thread, Postgres ledger, cooperative cancellation.
<code>report_pdf.py</code>	The branded HTML / PDF validation dossier.

9.2 Graph topology

The graph is a `StateGraph` over `AlphaResearchState`, compiled once per run. `prepare_data` executes once; everything from `train` to `quant_researcher` is the loop body.

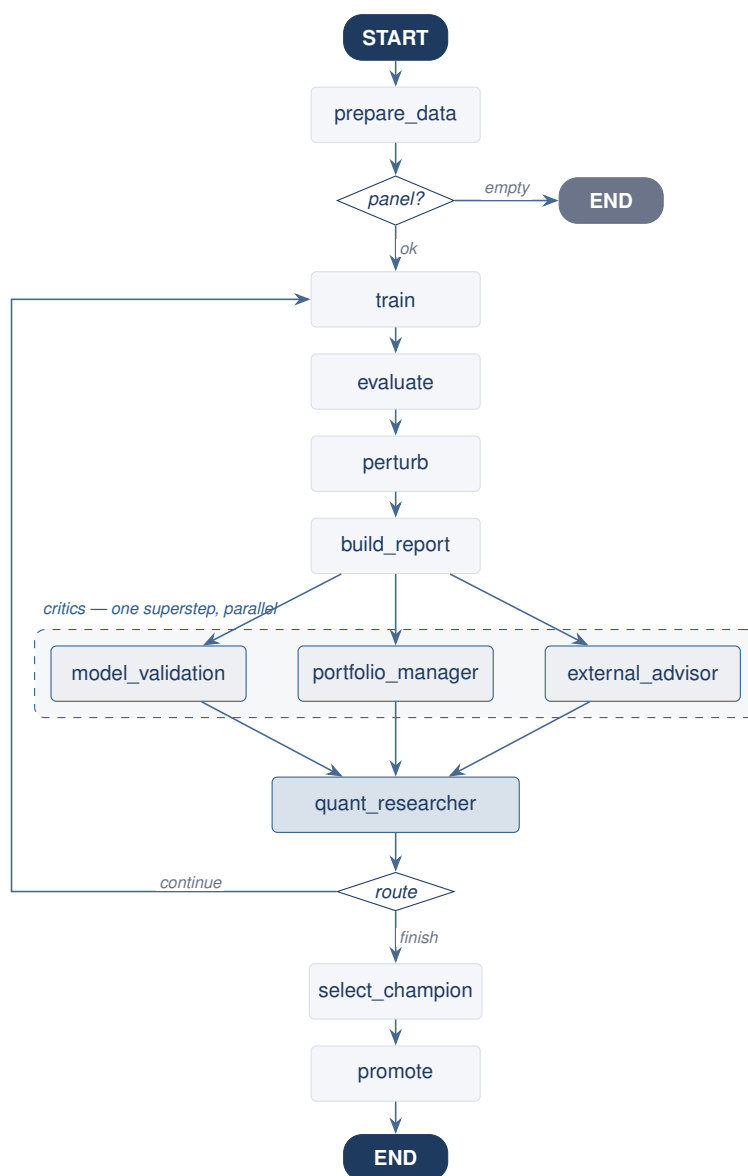


Figure 2. The research loop. The three critics fan out in one superstep and the researcher joins on all of them, so it synthesizes a complete set of critiques rather than reacting to whichever finished first.

9.3 State and reducers

AlphaResearchState is a TypedDict(total=False) following the committee’s convention. Exactly three channels accumulate, and they are exactly the channels written concurrently by the fan-out:

Channel	Reducer	Contents
iterations	operator.add	One validation report per completed round.
agent_notes	operator.add	The four personas’ output; written by three parallel nodes.
candidates	operator.add	Fitted models kept for champion selection and ensembling.

Everything else is last-write-wins, which is what a loop that overwrites its working specification each round requires. Heavyweight objects — the raw panel, the fitted candidates — live on the state as ordinary Python references: the graph runs in one process on one thread, so there is no serialization boundary, and re-deriving a multi-minute warehouse pull each round to satisfy a purity principle would be the wrong trade.

Node names may not collide with state keys

LangGraph rejects a graph whose node name matches a state key. The obvious names `report`, `validation` and `researcher` are all taken by channels, so the nodes are `build_report`, `model_validation` and `quant_researcher`. The failure is a hard `ValueError` at `add_node`, not a subtle one.

9.4 Routing and termination

`route_after_prepare` ends the run immediately when the warehouse yields no usable panel, rather than looping over emptiness. `route_iteration` has five independent stopping conditions, evaluated in priority order — cancellation; the iteration budget; a portfolio-manager `accept/reject`; the researcher declaring convergence or proposing nothing the validator accepted; and the guarded metric failing to improve for `patience` rounds. Each is recorded in `stop_reason`, so the report states why the run ended instead of leaving the reader to infer it from the round count. The recursion budget is $12 \times \text{max_iterations} + 20$, giving the eight-node body headroom.

9.5 The agents

Three of the four run on the desk provider (DeepSeek by default, any registry provider by request). The External Advisor runs on a *different* provider when the caller supplies a second key, so the outside view carries genuinely different priors; it falls back to the desk provider otherwise.

Agent	Returns	Power
Quantitative Researcher	<code>SpecPatch</code> — changed fields, rationale, a falsifiable hypothesis	Proposes the next specification
Model Validation	<code>ValidationVerdict</code> <code>pass warn fail</code> plus structured findings	A <i>fail</i> <i>veto</i> es promotion of that round
Portfolio Manager	<code>PMVerdict</code> <code>accept reject continue</code>	Ends the loop; final promotion gate
External Advisor	<code>AdvisorNote</code> — contrarian read + one orthogonal direction	Advisory only

Structured output uses the committee’s plumbing (`make_chat_model(...).with_structured_output`) with `retry-on-malformed`. Enum-like fields are typed `str`, not `Literal`, and normalized in code — the committee’s convention, adopted for the same reason: a model answering “PASS” should not lose an entire analysis to whole-response validation.

The deterministic path is not a stub

`MZQA_COMMITTEE_DISABLE_LLM=1`, `config["offline"]`, or simply no key puts the loop on a fully deterministic path: the researcher walks a pre-registered ladder of changes motivated by the incumbent pipeline’s known defects, and the three critics apply rule-based versions of their checklists. It is a usable no-token grid search, and it is what the test suite exercises.

9.6 Two hard boundaries

The LLM never runs code. Following FlowMind, each agent’s system prompt is a *lecture* over the `TrainingSpec` schema, generated from `describe_search_space()` so it cannot drift from the validator. The researcher returns a patch; `apply_patch` whitelists every key, range-clamps every value, drops hyperparameters foreign to the selected model family, and reports what it rejected back into the next round’s packet. A hallucinated field, or a `learning_rate` of 500, cannot reach a fit.

The LLM never sees instrument identity. Agent packets are built by an explicit allowlist and then swept: aggregate metrics and bucket statistics only, no per-name returns, no ranked name lists. The guard matches against the run’s *actual* universe rather than against anything ticker-shaped, and inspects identifier positions rather than prose — US symbols include `ALL`, `ON`, `MAX`, `GAP` and `REAL`,

so a text scan fires on almost every sentence an agent writes. The motivation is concrete: every current model's training data covers the evaluation window, so a packet naming the winners of 2023 would let an agent steer the specification toward them for reasons unrelated to the diagnostics.

9.7 Background job and ledger

A run is several fits plus a dozen LLM calls — past any request timeout — so it cannot use the blocking shape `/retrain` uses. `runner.start_run` registers the run, spawns a daemon thread, and returns a `run_id` immediately; the client polls. Rounds are persisted by an `on_iteration` hook as they complete, then re-persisted at the end once the critics' verdicts for each round exist. One active run per `(model_key, label)` is enforced both in-process and in the ledger, because two runs training the same model would race on the artifact path at promotion.

Tables (migration `136_quant_alpha_research.sql`, also created idempotently by `runner._ensure_tables`): `quant_alpha_research_run` — the header the UI polls, carrying status, live stage, champion, and the promotion decision — and `quant_alpha_research_iteration` — one row per round with the specification, the battery, the rating, both breakdowns, and all four verdicts.

9.8 Promotion

`select_champion` considers every round the Model Validation unit did not veto, *plus* an ensemble of them, and promotes only when the champion beats the incumbent on the guarded metric by a margin, the Portfolio Manager accepted, and the robustness rating is not *fragile*. Otherwise the artifact is written to a candidate path, the live model is untouched, and `promotion_reason` records why. When every round is vetoed the run still names its best attempt, explicitly flagged as not promotable — more useful than reporting no champion at all.

The incumbent comparison flatters the incumbent

The production model's recorded rank-IC comes from a single held-out block with no embargo; the champion's comes from a purged walk-forward. The two are not commensurable, and the promotion reason says so on every run.

10 REST API and frontend

Router `api/routers/quant.py` (under `/api/quant`), all handlers offloading qlib work to worker threads:

Endpoint	Purpose
GET <code>/backends</code>	Optimizer backends, risk models, alpha-model metadata.
POST <code>/alpha</code>	Expected returns for a universe (or the latest top- <i>N</i> cross-section).
POST <code>/risk</code>	Forward covariance / volatility / factor exposures.
POST <code>/optimize</code>	Portfolio optimization; <code>optimizer</code> selects native or any qlib backend.
POST <code>/backtest</code>	Walk-forward out-of-sample signal backtest with qlib <code>risk_analysis</code> .
POST <code>/retrain</code>	One-shot (re)train of a (market, horizon) model — the fast deterministic path, retained for the batch CLI.
POST <code>/research/start</code>	Start an agentic research run; returns a <code>run_id</code> immediately.
GET <code>/research/{id}</code>	Run header + every persisted round — the polling target.
GET <code>/research/latest</code>	Most recent run for a market/horizon, so the panel opens populated.
GET <code>/research/runs</code>	Run history, newest first.
POST <code>/research/{id}/cancel</code>	Cooperative cancel, checked between rounds.
GET <code>/research/{id}/report.pdf</code>	The rendered validation dossier.

The *Quant* tab (`QuantView.tsx`) exposes an optimizer-backend selector, risk-model and μ -source selectors, a live expected-returns table, an optimized-weights table, and a backtest panel.

11 Operations

- **qlib on Python 3.13.** No PyPI wheel exists for 3.13, so qlib is built from a bundled clone and installed non-editable, so `import qlib` resolves from any working directory (the C++ toolchain is required for the build). Lean runtime deps are pinned in `backend/requirements.txt` (`lightgbm`, `scikit-learn`, `cvxpy`, `ecos`, `dill`, `pyarrow`, ...).
- **Cold start / pre-warm.** The first alpha call for a jurisdiction builds the cross-section panel from the warehouse and then caches it (TTL via `QLIB_ALPHA_CACHE_TTL`); the walk-forward backtest is cached separately. Both are pre-warmed on startup so the first user request is served from cache.
- **Retraining.** The alpha model is (re)trained via the CLI: `python -m api.quant.qlib_alpha train --jurisdiction US`; artifacts are persisted per jurisdiction and hot-reloaded on file mtime.
- **Agentic research runs.** A run is a background job, not a request: `POST /api/quant/research/start` returns a `run_id` and the work continues on a daemon thread, with rounds persisted to `quant_alpha_research_{run,iteration}` as they complete. Apply migration `136_quant_alpha_research.sql` (or let `runner._ensure_tables` create the tables on first use). Run it headless with `python -m api.quant.research.graph --jurisdiction US --label forward_12m`, adding `--offline` to walk the deterministic ladder without spending tokens. Expect the warehouse pull to dominate the first minutes: the full US panel is the expensive step, and it is loaded once per run and reused by every round.
- **Tests.** `backend/api/quant/tests/test_quant.py` covers the optimizer backends, the structured covariance, the exact scoring fallback, the alpha blend, the IC re-split, and a DB-gated panel/train smoke test.

A lean, in-process qlib embed: one point-in-time panel, a learned alpha and a structured covariance, dual optimization, and a walk-forward backtest — wired into the scanner and the committee, and degrading cleanly to the prior behavior wherever a model or dependency is absent.